COMPUTER ENGINEERING
QUEZON CITY

# Final Project: Laboratory Borrowing System & Inventory Management System

**Course Code: CPE 009B** **Course Title: Object Oriented Programming 2**

## PROJECT INFORMATION

| Item | Details |
|---|---|
| Project Title | Laboratory Borrowing System |
| Section / Group No. | CPE21S4 / Group 3 |
| Instructor | Engr. Jimlord M. Quejado |
| Date Submitted | Oct 24, 2026 |

### Group Members

| # | Name (Surname, First Name M.I.) | Student No. | Role / Main Contribution |
|---|---|---|---|
| 1 | Acelajado, John Vince D. | 2516965 | PPT maker, UI Designer, Front-end |
| 2 | Bal, Kevin Ranz J. | 2513850 | Front-end, Back-end, Styling |
| 3 | Cuadera Paulie Jai | 2515947 | Data classes and file |
| 4 | Fabonan, Kurt Lorenz C. | 2511545 | Packaging and Back-end |
| 5 | Rebagos, Kian | 2513262 | Github Maker, Front-end |

## Intended Learning Outcomes Addressed

At the end of this activity, the students should be able to:

- Develop a solution to an industry-related problem (the group's chosen real-world problem) utilizing object-oriented analysis and design.
- Integrate the use of the object-oriented programming language, Python, with the design of a computer program by creating and using multiple classes.
- Create and present a program that solves a real-world problem using best coding practices in Python and implements Object-Oriented design principles such as encapsulation and composition

COMPUTER ENGINEERING

QUEZON CITY

## Objectives

**General Objective:** To develop a PyQt6 desktop application where it manages the borrowing and also approval of laboratory equipment from its student's request slip which professors decide if they are going to accept or deny. Then to the release and return of the items by its laboratory staff based on the inventory system .

**Specific Objectives:**

1. To implement a friendly Ui specifically for login, student dashboard, professor dashboard and also lab staff dashboard.
2. To provide a digital borrowing slip which includes the student id, student name, course & section, date needed, requested items and status.
3. To store and load all the data of the borrowing slips requested in a CSV file.
4. To manage the inventory system and also the release and return of the ID.
5. To make an executable file of the program project created.

## 1. INTRODUCTION

### 1.1 Background of the Project

In borrowing laboratory equipment inside school the current process starts when a student gets a borrowing slip from the laboratory staff or the designated office. The student completes the information needed and requests the professor's signature for approval. Once the professor signs the approved slip, the student will give their ID and hand it to the laboratory staff, who will check for approval before freeing up the equipment that the student requests and documenting its release and return.

The current method is not optimal because it is time consuming for a student to fill up form and wait for the professors about the approval of the slip, the student may also realize that the item they needed in laboratory is already borrowed or it is not available, and lastly when filing up the slip it can lead to erasures, missing quantities, signature also the incompleteness of it. This project solves the current problem by doing an application instead of manual paper slip which makes it more organized and saves all the records.

### 1.2 Statement of the Problem

*State the main problem in one sentence, then list the specific problems your application will solve. These should connect directly to your specific objectives.*

The manual process of filling up the borrower slip and managing the availability of laboratory equipment makes it difficult to monitor, track the borrower transactions and ensure the proper releasing and return of the laboratory equipment.

1. Some Student Requests are not getting stored and can't be seen by the professors and the laboratory admin.
2. Borrowing materials in the laboratory is still physical like using paper and pen to list things that are needed.
3. The lack of a standalone executable file to ensure smooth program distribution and installation.

### 1.3 Target Users

*Who will use the application? Describe each type of user and what they need to do with it.*

| User | Description | What They Do in the Application |
|---|---|---|
| Students | Requester | They fill out the materials and equipment they need for their laboratory equipment |
| Professors | Reviewer and Approver | They review and approve the request forms of the students that require laboratory equipment |
| Staff in the Laboratory Department | Approver | Accepting the approval and providing the requested laboratory equipment of the student/s |

### 1.4 Significance of the Project

This project benefits the members of TIP especially the Students who get a faster and more clearer way to request the laboratory equipment, check availability and the status of the request slip. Professors can review all the equipment requested by the students and approve it anytime. Lastly laboratory staff review the slip that the Professor approved and release the laboratory item in the slip. This project supports the SDG9(Industry, Innovation, and Infrastructure) by improving the manual borrowing slip process into a digital process which improves how the process of borrowing equipment is more convenient and this makes the workflow smooth and have a reliable record.

## 2. SCOPE AND DELIMITATIONS

### 2.1 Scope

*List every feature your application will have. Be specific: "Add, edit, and delete records" is better than "Manage records". Every feature listed here must appear in your results.*

| # | Feature | Description | Window / Page |
|---|---|---|---|
| 1 | Login System | It contains the student's ID, name, year level, course and section, program, date needed, time, professor, material name, and quantity, as well as two buttons labeled “Add Materials to Request' and “Add Item” | Login Window |
| 2 | Student Dashboard | It features username and password input fields, a 'Show Password' checkbox to reveal the entered password, and two buttons: “Clear”, which resets the inputs, and “Login”, which directs you to the dashboard. | Dashboard Window |

| | | | |
|---|---|---|---|
| 3 | Professor Dashboard | It displays a table grid on the dashboard and displays the requested form of the student/s for the laboratory equipment/s where it chooses whether to accept or decline the request form and it needs to click the table that shows all the information and request of the student/s before accepting or declining it. If the instructor declines the request, it is required to input a reason on the textbox for it before declining, or else it will display a pop up message that it needs to input a reason before the decline proceeds. | Dashboard Window |
| 4 | Laboratory Staff Dashboard | [ ] | [ ] |
| 5 | Inventory Management System | [ ] | [ ] |

### 2.2 Delimitations

- The application will not use database (MySQL, MySQLite3, & SQLite)
- The application is limited to TIP Students, TIP Professors and TIP Laboratory Staff only
- The account switching feature will demonstrate the three user roles but will not provide a complete authentication and user registration.
- The application will not physically collect or verify the student's ID, The laboratory staff will remain responsible for checking and holding the physical ID.
- The application will run on a single desktop computer only and will not connect or synchronize the information/data that it receives from the app when executing it across multiple computers.
- The application will not provide automated email or SMS notification. The notifications like announcement from the Professor Dashboard will be displayed only in the application.
- The application will not automatically detect the damage, missing, or incomplete materials. The Laboratory Staff will remain responsible to inspect, return items and enter relevant information.
- The Project will focus on developing and demonstrating the prototype rather than applying it a full operation of the system across several laboratories.

### 2.3 Minimum Technical Requirements

*Your project must meet every requirement below. Tick each one when it is done, and show where it is met in Sections 4 and 5.*

| Done | Requirement |
|---|---|
| [ /] | A desktop GUI built with PyQt6 |
| [ /] | At least four classes of your own, organized in at least three Python files (modules) |
| [ /] | Encapsulation, abstraction, inheritance, and polymorphism, each applied at least once |
| [ /] | Input validation with clear feedback to the user |

| | |
|---|---|
| [ ] | Data saved to and loaded from a file (e.g., .txt, .csv, or .json) |
| [ ] | At least two windows or pages, with navigation between them |
| [ ] | A menu bar or toolbar, and at least one dialog |
| [/ ] | Consistent styling using Qt Style Sheets (QSS) |
| [ ] | The application packaged as a working Windows .exe |

## 3. EXPECTED OUTPUT

### 3.1 System Overview

*Describe the finished application in one paragraph: what the user sees first, the main windows or pages, and the typical flow from start to finish.*

The login window is seen by the user first. Then, if they are a student, the student window opens for the student, which allows them to request materials and equipment from the laboratory department. If they are a professor, the professor window opens for the professor, which allows them to approve or disapprove lab requests from the student and add notes just in case.

### 3.2 Tools and Technologies

| Tool | Version | Purpose |
|---|---|---|
| Python | 3.12 | Programming language |
| PyQt6 | 6.11.0 | Graphical user interface |
| Visual Studio Code | 1.94.2 | Code editor |
| PyInstaller | 6.22.3 | Packaging as a Windows .exe |
| [Other library or tool] | [ ] | [ ] |

### 3.3 Project File Structure

*Show the folders and files of your project and what each file contains.*

```
[project_folder]/
├── main.py              # [entry point: creates QApplication]
├── [models.py]          # [data classes]
├── [main_window.py]     # [main window and navigation]
├── [dialogs.py]         # [custom dialogs]
├── [style.qss]          # [application theme]
└── [data.json]          # [saved data]
```

### 3.4 Class Diagram

*Insert a UML class diagram showing each class with its main attributes and methods, and the relationships between classes (inheritance arrows and composition / "has-a" lines). You may draw it with draw.io or Lucidchart*

*[Insert screenshot or diagram here]*

Figure 1. Class diagram of [Project Title]

### 3.5 Expected Output

*Describe what the user gets from the application: the screens they will use, the files it creates, and the final deliverables of the project.*

| Deliverable | Description |
|---|---|
| Working application | [Source code that runs with python main.py] |
| Windows executable | [Packaged .exe in a zip file] |
| Project report | [This document] |
| Presentation | [Slides and live demonstration for the final presentation] |

## 4. APPLICATION OF OBJECT-ORIENTED PROGRAMMING PRINCIPLES

*This is the most important section of the report. For each principle, name the class and method where it is applied, paste a short code excerpt (5 to 20 lines, not the whole file), and explain in your own words why this design is better than the alternative. The full code goes in Appendix A.*

### 4.1 Classes and Objects

*List every class you wrote.*

| Class | File | Inherits From | Responsibility |
|---|---|---|---|
| LoginWindow | models.py | object | Stores one record and validates its own data |
| Professor Dashboard | profui.py | QWidget | Shows the Dashboard GUI for the professors |
| Student Dashboard | student.py | QWidget | Shows the Dashboard GUI for the students |
| [ ] | [ ] | [ ] | [ ] |

### 4.2 Encapsulation

*Show a class that keeps its data private (for example, attributes starting with an underscore) and exposes it only through methods or properties that check the data.*

**Where applied:** [Class name, method or property name]

```
# [Paste a short code excerpt here]
```

**Explanation:** [Write here.]

### 4.3 Abstraction

*Show how a class hides complex details behind simple methods (for example, a save() method that hides how the file is written), or an abstract base class that defines what subclasses must do.*

**Where applied:** [Class name, method name]

```
# [Paste a short code excerpt here]
```

**Explanation:** [Write here.]

### 4.4 Inheritance

*Show a class of your own that inherits from another class. Inheriting from a PyQt6 class (QWidget, QDialog, QMainWindow) counts, but also include one inheritance between two of your own classes if possible.*

**Where applied:** [Child class → Parent class]

```
# [Paste a short code excerpt here]
```

**Explanation:** [Write here.]

### 4.5 Polymorphism

*Show a method that behaves differently depending on the object, for example an overridden method (__str__, accept(), closeEvent()) or a method with the same name in several classes that is called in one loop.*

**Where applied:** [Class names, method name]

```
# [Paste a short code excerpt here]
```

**Explanation:** [Write here.]

### 4.6 Composition and Modularization

*Show where one object contains other objects ("has-a"), such as a main window that has a data manager, and explain how splitting the code into modules made it easier to build and test.*

**Where applied:** [Class that contains → classes it contains]

```
# [Paste a short code excerpt here]
```

**Explanation:** [Write here.]

### 4.7 GUI Concepts Applied

*Indicate which course topics your project uses and where. Use the class and method names from your code.*

| Course Topic | Used? (Yes / No) | Where in the Project |
|---|---|---|
| Signals and slots | [ ] | [ ] |
| Event binding and callbacks | [ ] | [ ] |
| Forms and input validation | [ ] | [ ] |
| Classes and modularization in GUI | [ ] | [ ] |
| File I/O integration | [/] | [ ] |
| Dialogs, menus, and toolbars | [ ] | [ ] |
| Multi-window and navigation | [ ] | [ ] |
| Styling GUIs (QSS) | [/] | prof.qss and student.qss |
| Packaging as an .exe | [ ] | [ ] |

## 5. RESULTS AND DISCUSSION

### 5.1 Application Screens

*Show a screenshot of every main window, page, and dialog. Below each one, write a caption and one or two sentences explaining what the user can do there. Add as many figures as needed.*

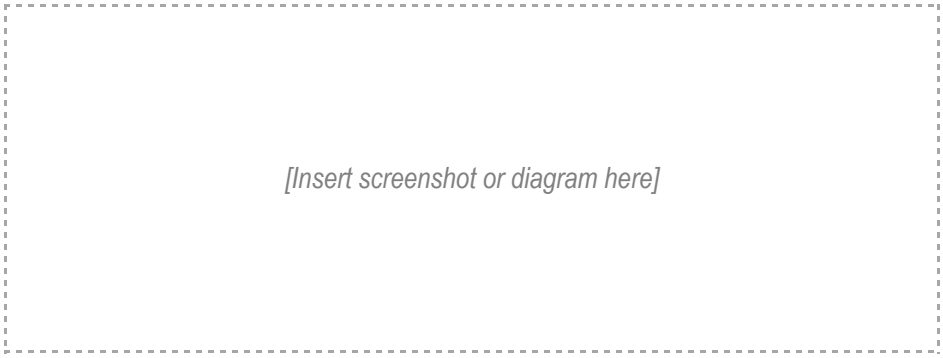



Figure 2. [Login window] - [what it shows and does]

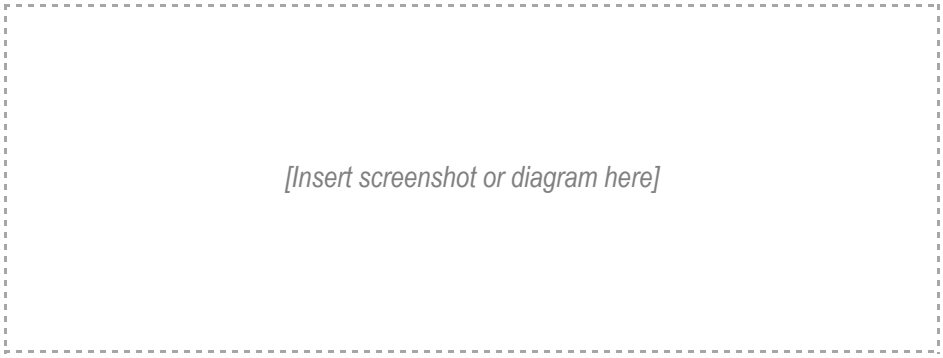



Figure 3. [Main window] - [what it shows and does]

### 5.2 Test Results

*Test every feature from Section 2.1, including invalid input. Record what actually happened. Aim for at least ten test cases.*

| # | Feature Tested | Input / Action | Expected Result | Actual Result | Pass / Fail |
|---|---|---|---|---|---|
| 1 | [Login] | [Wrong password] | [Warning message; fields cleared] | [ ] | [ ] |
| 2 | [Login] | [Correct password] | [Main window opens] | [ ] | [ ] |
| 3 | [ ] | [ ] | [ ] | [ ] | [ ] |
| 4 | [ ] | [ ] | [ ] | [ ] | [ ] |
| 5 | [ ] | [ ] | [ ] | [ ] | [ ] |

### 5.3 Discussion

*Explain your results. Did the application meet each specific objective? Refer to your figures and test results ("As shown in Figure 3 and Test 5, ..."). Do not only describe the screens; explain what the results mean.*

| Specific Objective | Met? (Yes / Partly / No) | Evidence (Figure or Test No.) |
|---|---|---|
| [Objective 1] | [ ] | [ ] |
| [Objective 2] | [ ] | [ ] |
| [Objective 3] | [ ] | [ ] |

[Write your discussion here.]

### 5.4 Challenges Encountered

*Describe real problems you met while building the project and how you solved them. These show what you learned.*

| Challenge | Cause | Solution |
|---|---|---|
| [e.g., The .exe could not find style.qss] | [The file was not bundled] | [Added --add-data and a path based on __file__] |
| [ ] | [ ] | [ ] |

## 6. CONCLUSION AND RECOMMENDATIONS

### 6.1 Conclusion

*Summarize what the group built and whether the objectives were achieved. Do not introduce new results here. One to two paragraphs.*

[Write here.]

### 6.2 Recommendations

*Suggest specific improvements for a future version, based on your limitations and test results.*

- [Recommendation 1]
- [Recommendation 2]

## 7. REFERENCES

*List every source you used (documentation, tutorials, books, and AI tools approved by your instructor) in APA 7th edition format, in alphabetical order. Disclose any AI assistance as required by the course AI policy.*

[Author, A. A. (Year). Title of work. Publisher or Site. URL]

[Riverbank Computing. (n.d.). PyQt6 reference guide. https://www.riverbankcomputing.com/static/Docs/PyQt6/]

## APPENDICES

### Appendix A. Source Code

*Paste the complete code of every file, one file per sub-section, in the same order as your file structure in Section 3.3. Use a monospace font (Consolas or Courier New, 8 to 9 pt) and keep the indentation. Include your style sheet and any data file formats.*

#### A.1 main.py

```
# [Paste the full contents of main.py here]
```

#### A.2 [file_name.py]

```
# [Paste the full contents here]
```

#### A.3 [style.qss]

```
/* [Paste the full contents here] */
```

### Appendix B. User Guide

*Write step-by-step instructions that a first-time user can follow, from opening the .exe to completing the main task. Include screenshots where helpful.*

1. [Extract [AppName].zip and double-click [AppName].exe.]
2. [Log in using ...]
3. [ ]

### Appendix C. Project Files and Links

| Item | Link or File Name |
|---|---|
| Source code repository (e.g., GitHub) | [ ] |
| Packaged application (.zip with .exe) | [ ] |
| Demo video (3 to 5 minutes) | [ ] |
| Presentation slides | [ ] |

### Appendix D. Work Distribution

*Describe what each member actually did. This must agree with the roles in Project Information.*

| Member | Tasks Completed | Files / Classes Written | Contribution (%) |
|---|---|---|---|
| [ ] | [ ] | [ ] | [ ] |
| [ ] | [ ] | [ ] | [ ] |
| [ ] | [ ] | [ ] | [ ] |
| [ ] | [ ] | [ ] | [ ] |

## EVALUATION CRITERIA

*For reference: this is how the project report and application will be graded. The final presentation and demonstration are graded separately.*

| Criteria | Description | Points |
|---|---|---|
| Problem and Objectives | Clear real-world problem; specific, measurable objectives | 10 |
| Scope and Output | Realistic scope; features, delimitations, and deliverables well defined | 10 |
| Functionality | All features in the scope work correctly, including input validation and file storage | 20 |
| Application of OOP Principles | Correct and meaningful use of encapsulation, abstraction, inheritance, polymorphism, and composition, clearly explained | 25 |

| | | |
|---|---|---|
| GUI Design and Usability | Consistent styling, clear navigation, helpful feedback and dialogs | 15 |
| Results and Discussion | Complete screenshots and test cases; objectives evaluated with evidence | 10 |
| Documentation and Code Quality | Complete appendices, readable and commented code, working .exe | 10 |
| Total | | 100 |